M04 MATERI 1

1. **TUJUAN**

   CPMK : Mahasiswa mampu merancang dan mengembangkan aplikasi web front-end yang interaktif dan efisien dengan memanfaatkan framework modern seperti ReactJS,
   Sub-CPMK :

   a. Mahasiswa memahami konsep dasar ReactJS, termasuk komponen, props, dan state dalam membangun aplikasi front-end yang dinamis.

2. **DURASI WAKTU**

   1 pertemuan x 3 jam

3. **DASAR TEORI**

   React JS

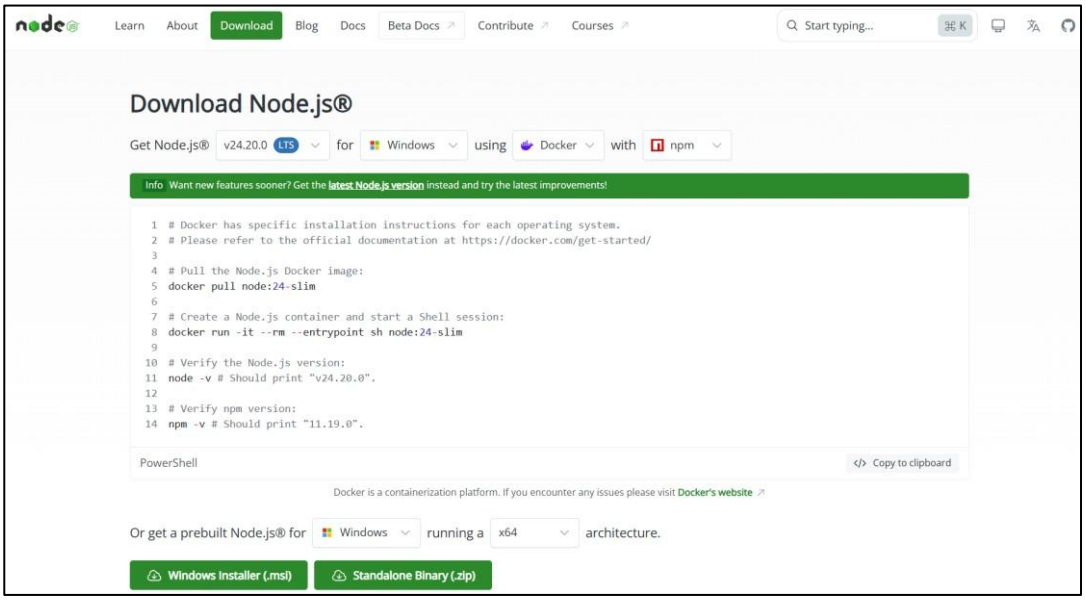
4. Percobaan

A. Membuat Project React JS Menggunakan Vite

1. Pastikan bahwa kita melakukan instalasi node.js terlebih dahulu
   - Cari “Download Node.js” di browser
   - Klik link seperti pada gambar dibawah ini:



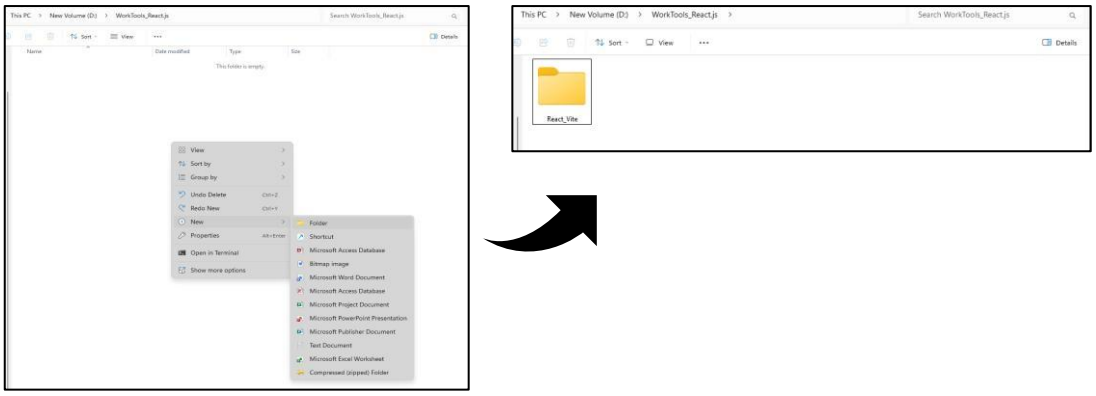
   - Sebelum melakukan instalasi, pastikan Anda menggunakan Node.js versi terbaru.
   - Selanjutnya, klik tombol “Windows Installer (.msi)” untuk mengunduh file instalasi.



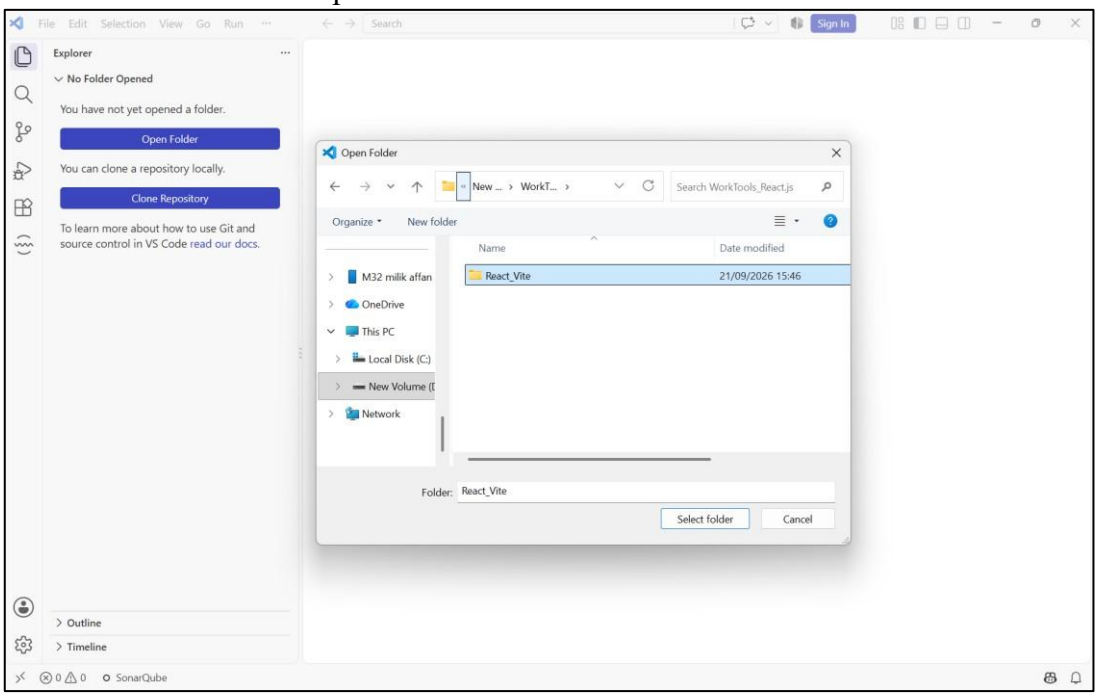
   - Buka Command Prompt (CMD) pada laptop.
   - Ketik perintah node --version, lalu tekan Enter untuk mengecek versi Node.js yang terpasang.

```
C:\Users\lab_mi2>node --version
v24.21.0
```

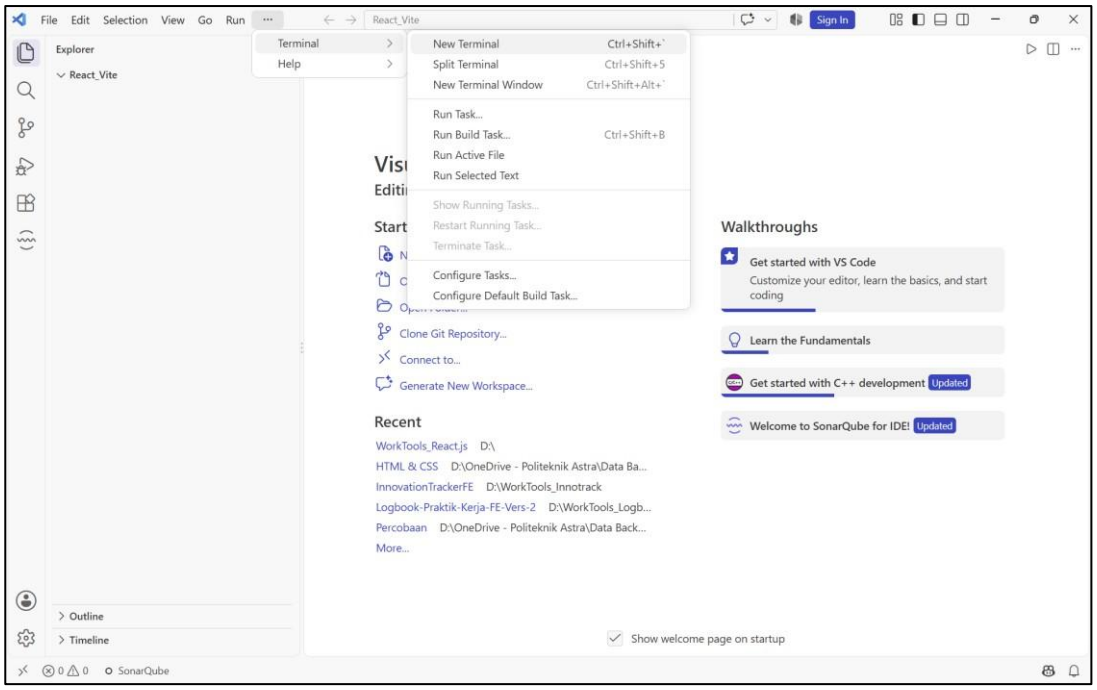
2. Setelah Node.js telah diinstal, buat folder baru pada file explorer. Pembuatan folder dapat dilakukan manual seperti membuat folder pada umumnya.



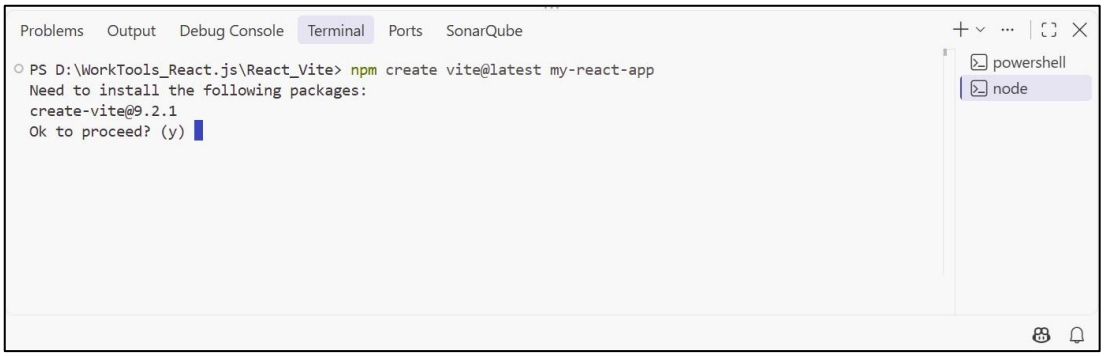
3. Buka VS Code dan Open Folder untuk bisa dibuka di vs code kita



4. Buka teminal pada vs code



5. Tuliskan pada terminal npm create vite@latest my-react-app. ***My-react-app*** adalah nama project yang ingin kita buat. Kita dapat mengubah Namanya sesuai dengan nama project yang kita mau.



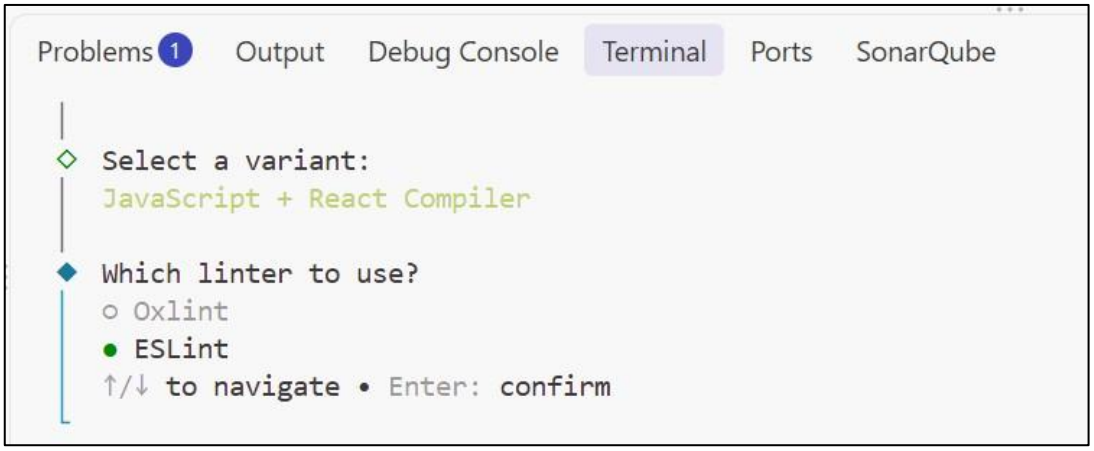
6. Setelah menjalankan perintah npm create vite@latest, Vite akan menampilkan beberapa pilihan *framework* yang dapat digunakan untuk membuat aplikasi web, seperti Vanilla, Vue, React, Svelte, dan lainnya. Pada praktikum ini, *framework* yang digunakan adalah React. Ketika "y" lalu Enter, kemudian tunggu hingga daftar pilihan muncul seperti pada gambar. Gunakan tombol panah ke bawah (↓) untuk memilih React.

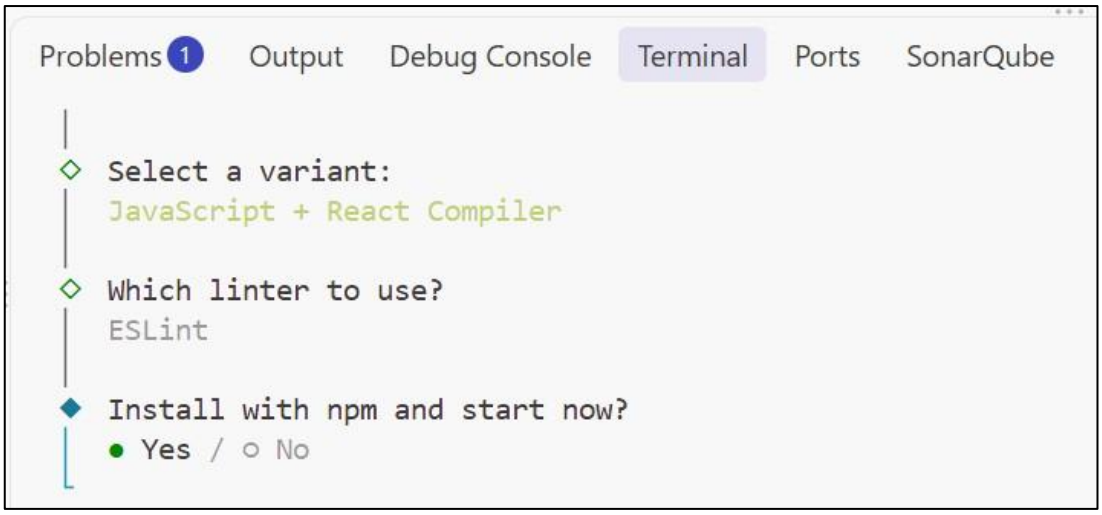


7. Kemudian pilih JavaScript + React Compiler, dan tekan Enter

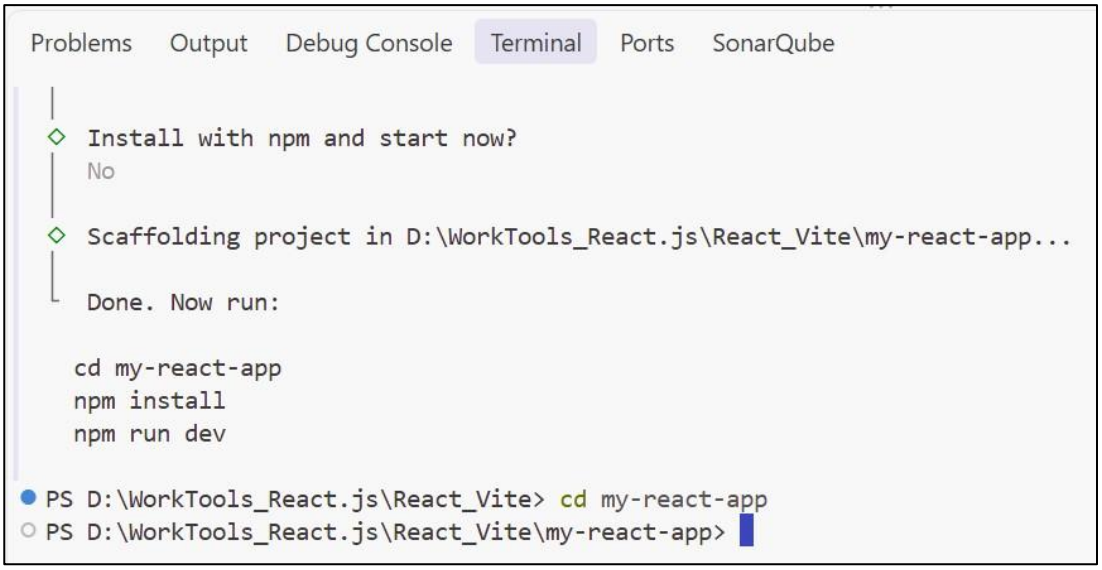


8. Kemudian pilih EsLint





9. Masuk ke dalam folder yang telah kita buat dengan cara mengetikkan *cd myreact-app* pada terminal



10. Setelah masuk ke dalam folder proyek, lakukan instalasi npm dengan mengetikkan *npm install*

```
Problems  Output  Debug Console  Terminal  Ports  SonarQube

◇ Scaffolding project in D:\WorkTools_React.js\React_Vite\my-react-app...
└ Done. Now run:

  cd my-react-app
  npm install
  npm run dev

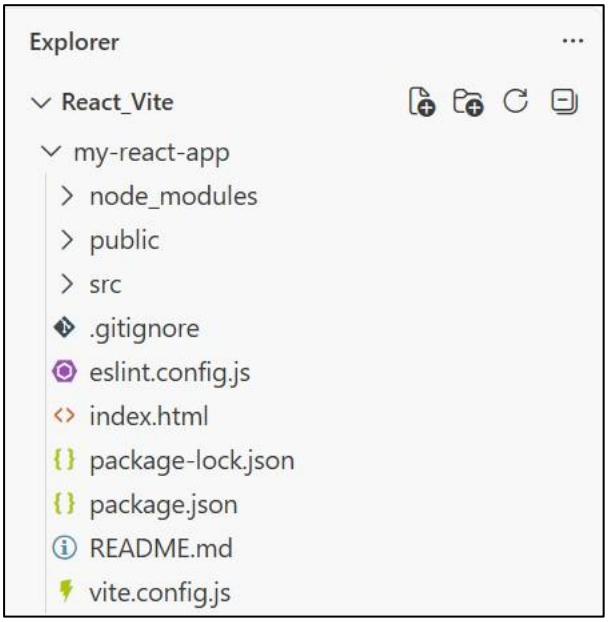
PS D:\WorkTools_React.js\React_Vite> cd my-react-app
PS D:\WorkTools_React.js\React_Vite\my-react-app> npm install

added 145 packages, and audited 146 packages in 25s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS D:\WorkTools_React.js\React_Vite\my-react-app>
```

11. Berikut adalah struktur dari folder proyek



- **Folder Utama: my-react-app/**
  Ini adalah folder utama untuk project React kita. Di dalamnya, kita akan menemukan semua file dan subfolder yang diperlukan untuk pengembangan aplikasi.
- **Folder node_modules/**
  Folder ini berisi semua dependensi yang diinstal menggunakan npm. Setiap paket atau library yang Anda tambahkan ke proyek (seperti React, Vite, dan lainnya) akan disimpan di sini. Ini adalah bagian penting dari aplikasi karena

semua pustaka eksternal yang digunakan dalam aplikasi Anda berada di dalam folder ini.

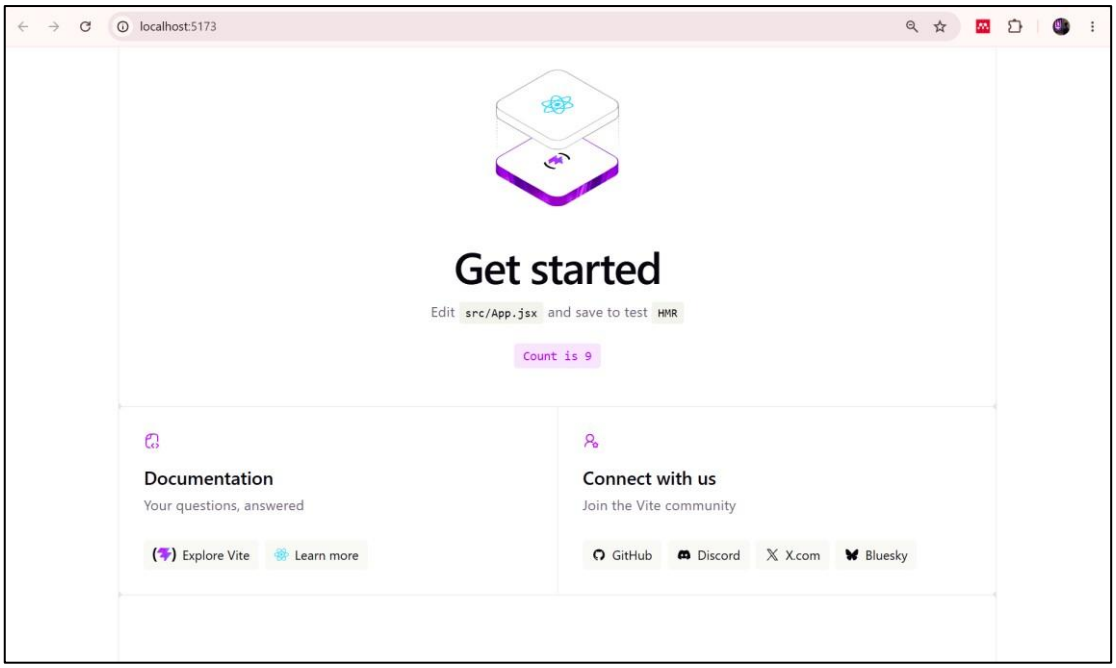
- **Folder: public/**
  Folder ini berisi file statis yang akan disajikan langsung oleh server tanpa melalui proses build. File yang umum ditemukan di sini meliputi: File gambar atau aset lainnya yang mungkin digunakan dalam aplikasi. File-file dalam folder ini akan dapat diakses langsung melalui URL.
- **Folder: src/**
  Ini adalah folder utama di mana semua kode sumber aplikasikitaberada. Folder ini berisi komponen, gaya, dan aset yang dibutuhkan untuk aplikasi React. Subfolder didalamnya:
- assets/: Folder ini digunakan untuk menyimpan file aset seperti gambar atau ikon.
- App.css: File CSS yang digunakan untuk mendefinisikan gaya komponen App.jsx. Di sini, kita bisa menambahkan styling untuk elemen dalam aplikasi kita.
- App.jsx: Komponen utama aplikasi yang berisi struktur dasar UI aplikasi kita. Biasanya, kita akan mengimpor komponen lain di sini.
- index.css: File CSS yang mungkin digunakan untuk gaya global yang diterapkan di seluruh aplikasi. Kita bisa menambahkan aturan CSS umum di sini.
- main.jsx: File entry point aplikasi. Di sini, kita mengimpor React dan ReactDOM serta merender komponen App ke dalam elemen DOM dengan ID root.
- **File .gitignore**
  File ini digunakan untuk menentukan file dan folder yang tidak ingin kita sertakan dalam version control (Git)
- **File eslint.config.js**
  Ini adalah file konfigurasi untuk ESLint, yang membantu mendeteksi kesalahan potensial dan menjaga konsistensi dalam kode JavaScript kita. Di sini, kita dapat menyesuaikan aturan linting yang diterapkan pada proyek.
- **File index.html**
  File HTML utama untuk aplikasi kita. Vite akan menyisipkan aplikasi React ke dalam file ini.

- **File package.json**
  File ini berisi metadata tentang project kita, termasuk nama, versi, dan daftar dependensi yang digunakan.
- **File README.md**
  Dokumen ini memberikan informasi tentang proyek kita, cara menjalankan, dan penggunaan lainnya. Ini berguna untuk kolaborator atau pengguna lainnya.
- **File vite.config.js**
  File konfigurasi untuk Vite. Ini berisi pengaturan yang digunakan oleh Vite dalam menjalankan aplikasi kita dan kita dapat menyesuaikan berbagai pengaturan seperti alias, plugin, dan optimasi build di sini.

12. Kemudian coba untuk menjalankan proyek dengan mengetikan *npm run dev* pada terminal



13. Buka browser dan ketikkan http://localhost:5173/

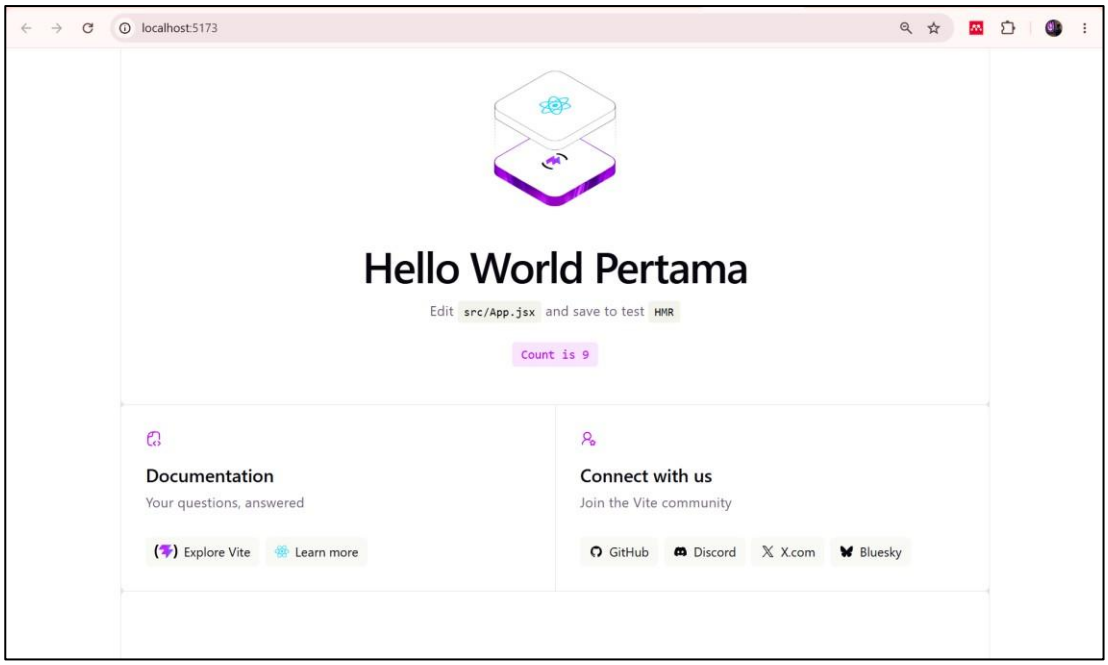


B. Membuat Hello World

1. Buka file App.jsx pada sub folder src di bagian explorer vs code kita. Kemudian ketikkan hello world seperti di bawah ini

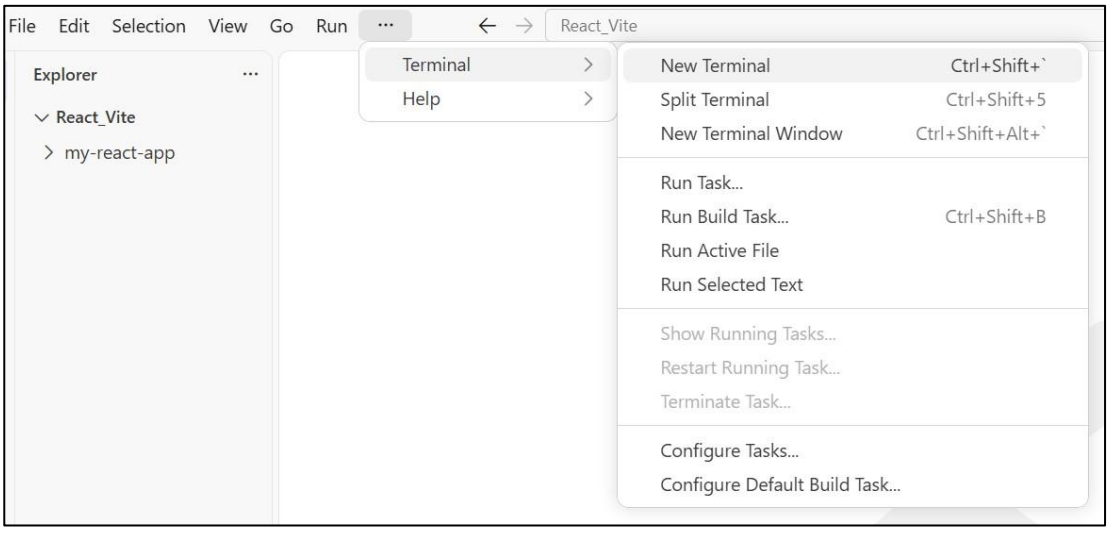


2. Hasil bisa dilihat langsung pada browser tanpa kita melakukan apapun, karena Vite telah memfasilitasi hal tersebut.

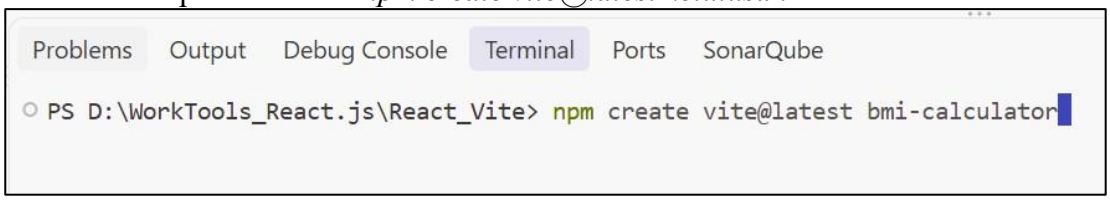


C. Membuat Program Mengecek Kategori BMI (Body Mass Index) dengan React JS

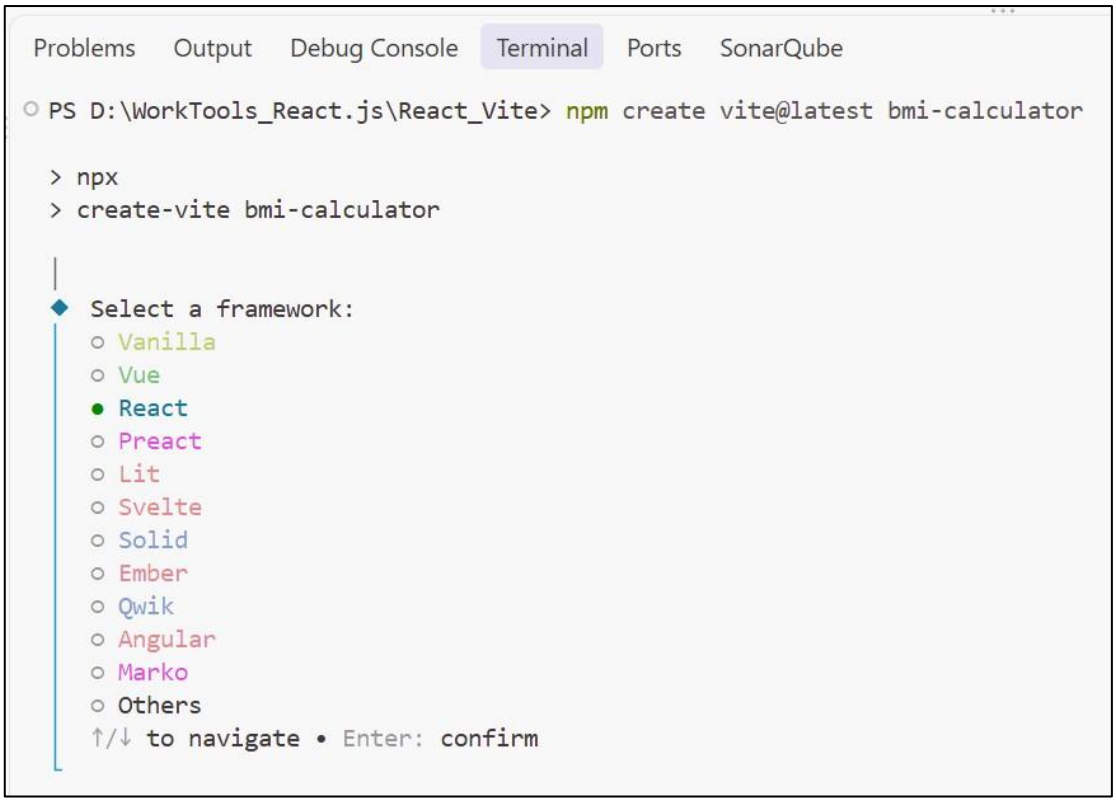
1. Membuat struktur folder proyek. Buka terminal baru dengan cara memilih menu Terminal > New Terminal



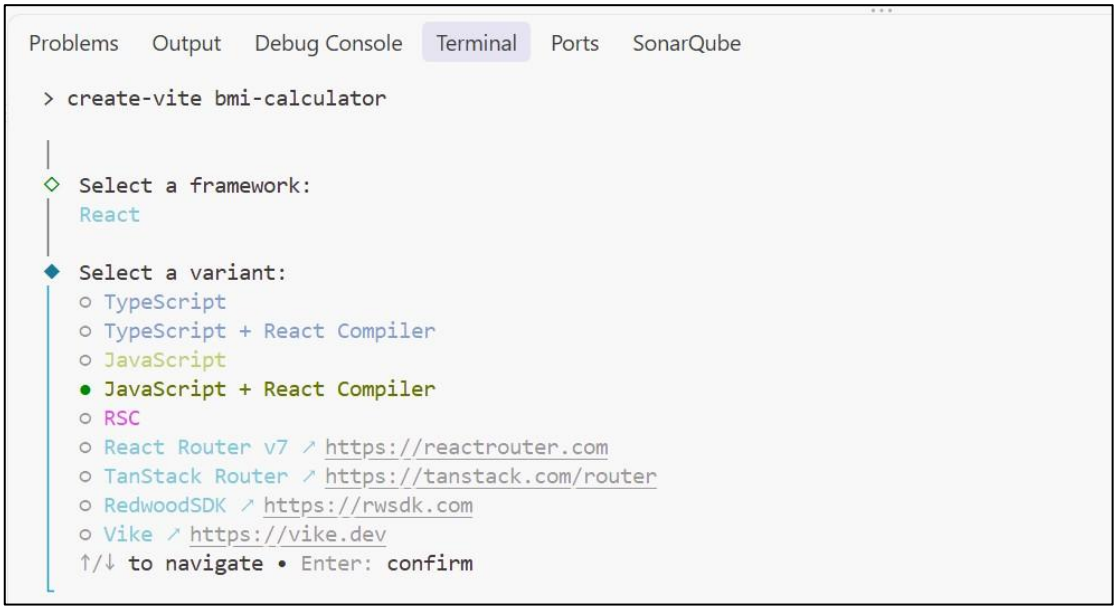
2. Ketikkan pada terminal *npm create vite@latest kelulusan*



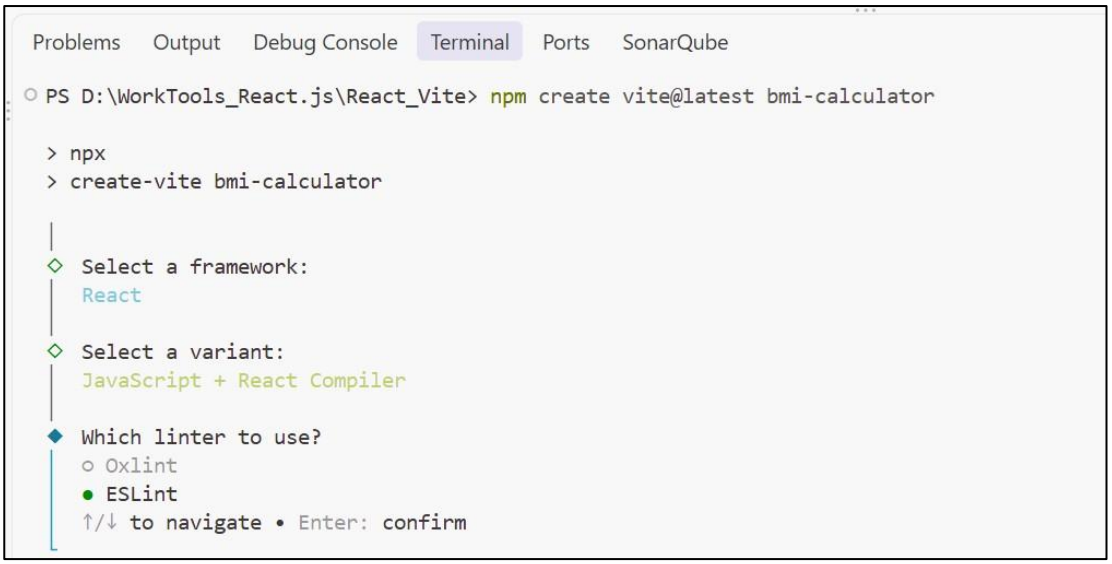
3. Kemudian pilih react



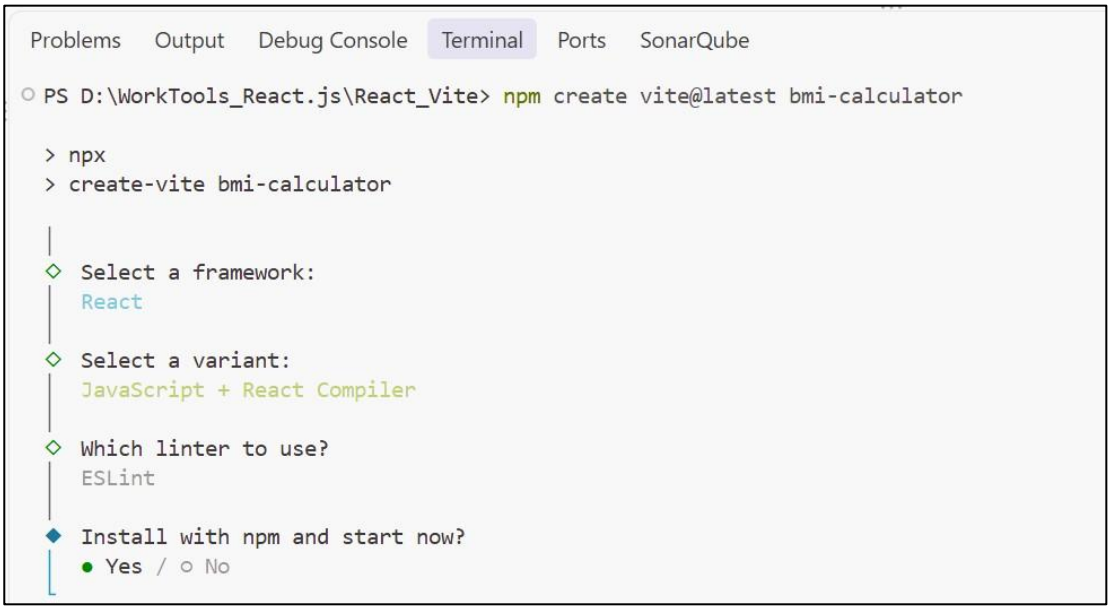
4. Kemudian pilih javascript + react compiler



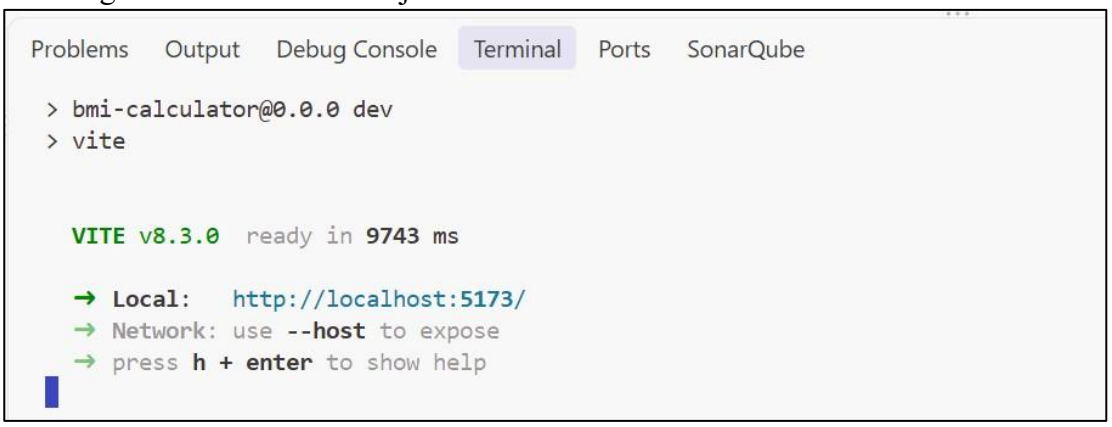
5. Kemudian pilih Eslint



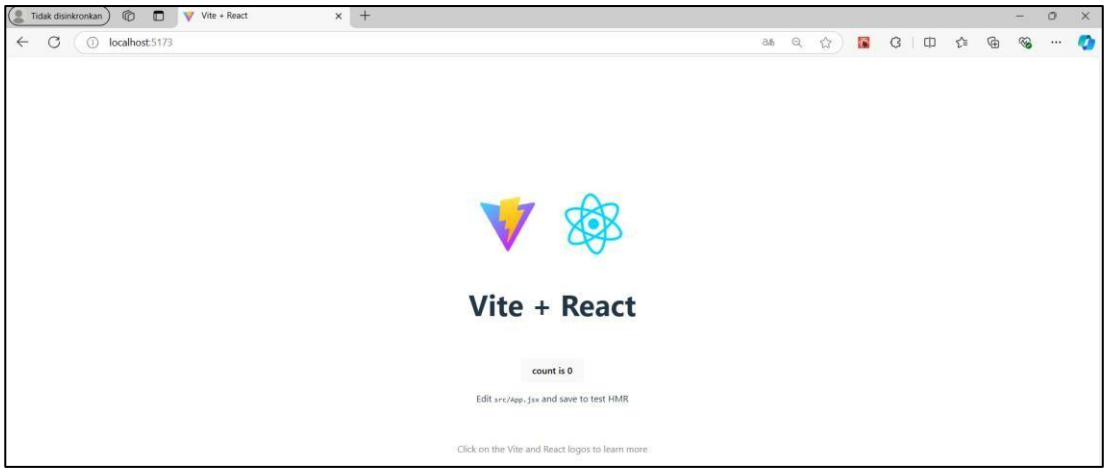
6. Kemudian pilih Yes



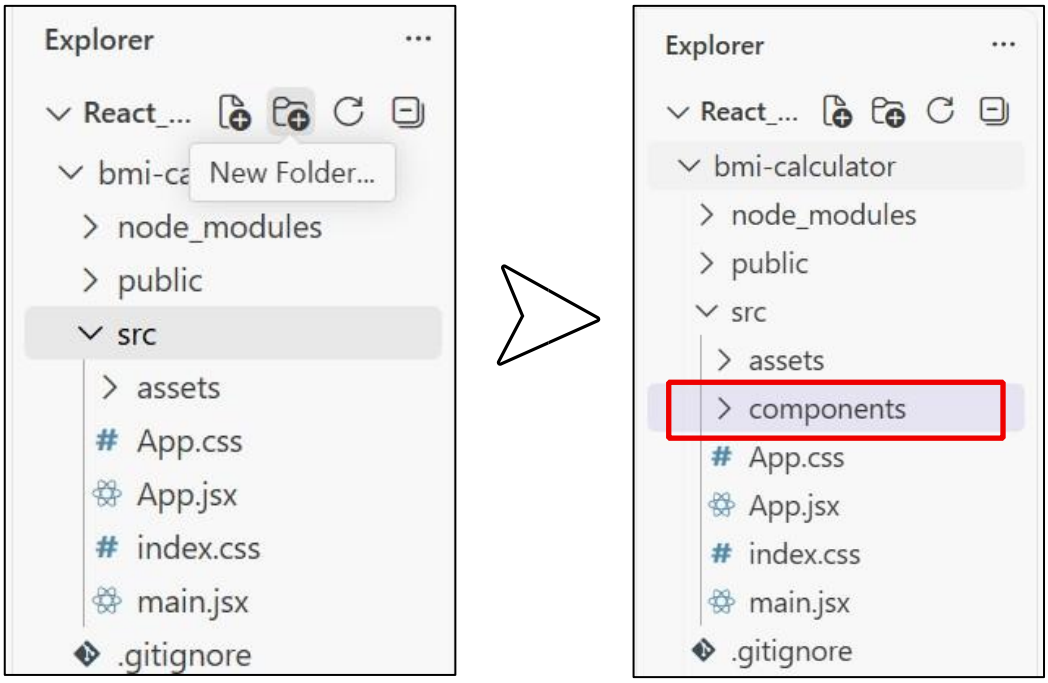
7. Program akan otomatis berjalan



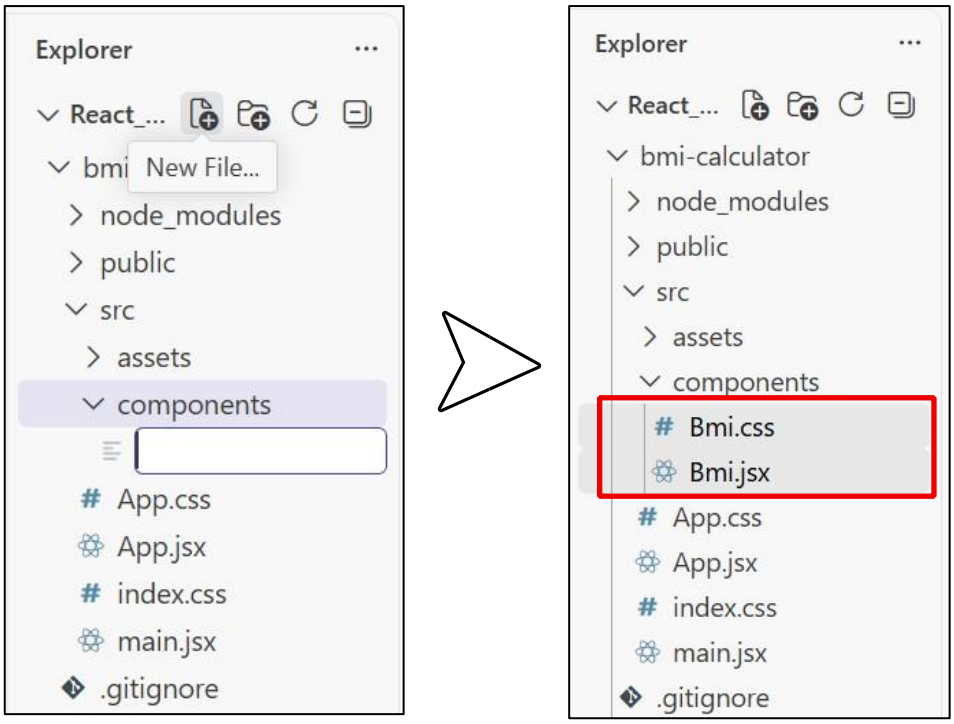
8. Kita dapat melihat hasilnya pada browser dengan mengetikkan alamat pada urlnya *http://localhost:5173/* . Contoh hasilnya adalah sebagai berikut.



9. Untuk bisa membuat program Mengecek Kategori BMI, kita harus mempunyai beberapa file yaitu komponen yang ada pada Bmi.jsx, Bmi.css dan App.jsx. Pada folder src buat folder baru dengan nama components. Arahkan cursor ke folder src > Klik New Folder



10. Kemudian buat file baru dengan nama Bmi.jsx dan Bmi.css Struktur folder yang harus ada dapat dilihat seperti berikut



11. Pada file Bmi.css ketikkan code sebagai berikut, disini kita ingin memberikan style kepada setiap input yang ada

```css
/* src/components/Bmi.css */
.container {
  margin: 50px auto;
  padding: 20px;
  max-width: 400px;
  text-align: center;
  border: 1px solid #ccc;
  border-radius: 8px;
}

.form {
  display: flex;
  flex-direction: column;
  align-items: center;
}

.label {
  margin-bottom: 10px;
  width: 100%;
  text-align: left;
}

.input {
  padding: 10px;
  margin-top: 5px;
  margin-bottom: 10px;
  width: 100%;
  box-sizing: border-box;
}

.button {
  padding: 10px 20px;
  background-color: #007bff;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  width: 100%;
}

.result {
  margin-top: 20px;
  font-size: 18px;
  padding: 10px;
  background-color: #f8f9fa;
  border-radius: 4px;
}
```

12. Kemudian pada Bmi.jsx, kita akan menulisan code sebagai berikut

```jsx
// src/components/Bmi.jsx
import { useState } from 'react';
import './BmiCalculator.css';

const BmiCalculator = () => {
  // 1. Inisialisasi state untuk berat, tinggi, dan hasil kalkulasi
  const [berat, setBerat] = useState('');
  const [tinggi, setTinggi] = useState('');
  const [hasil, setHasil] = useState(null);

  // 2. Handler saat form disubmit
  const handleSubmit = (e) => {
    e.preventDefault();

    const beratVal = parseFloat(berat);
    const tinggiVal = parseFloat(tinggi) / 100; // Konversi cm ke meter

    // Validasi input
    if (isNaN(beratVal) || isNaN(tinggiVal) || tinggiVal <= 0 || beratVal <= 0) {
      setHasil({ bmi: 0, status: 'Input nilai berat dan tinggi tidak valid!' });
      return;
    }

    // Perhitungan nilai BMI: Berat (kg) / (Tinggi (m) * Tinggi (m))
    const bmiVal = (beratVal / (tinggiVal * tinggiVal)).toFixed(1);
    let statusKategori = '';

    // Logika Pengkondisian (Conditional Branching)
    if (bmiVal < 18.5) {
      statusKategori = 'Kurang Berat Badan (Underweight)';
    } else if (bmiVal >= 18.5 && bmiVal <= 24.9) {
      statusKategori = 'Berat Badan Normal (Ideal)';
    } else if (bmiVal >= 25 && bmiVal <= 29.9) {
      statusKategori = 'Kelebihan Berat Badan (Overweight)';
    } else {
      statusKategori = 'Obesitas';
    }

    // Simpan hasil ke dalam state
    setHasil({ bmi: bmiVal, status: statusKategori });
  };

  return (
    <div className="container">
      <h2>Kalkulator Kategori BMI</h2>
      <form onSubmit={handleSubmit} className="form">
        <label className="label">
          Berat Badan (kg):
          <input
            type="number"
            className="input"
            value={berat}
            onChange={(e) => setBerat(e.target.value)}
            placeholder="Masukkan berat badan"
          />
        </label>

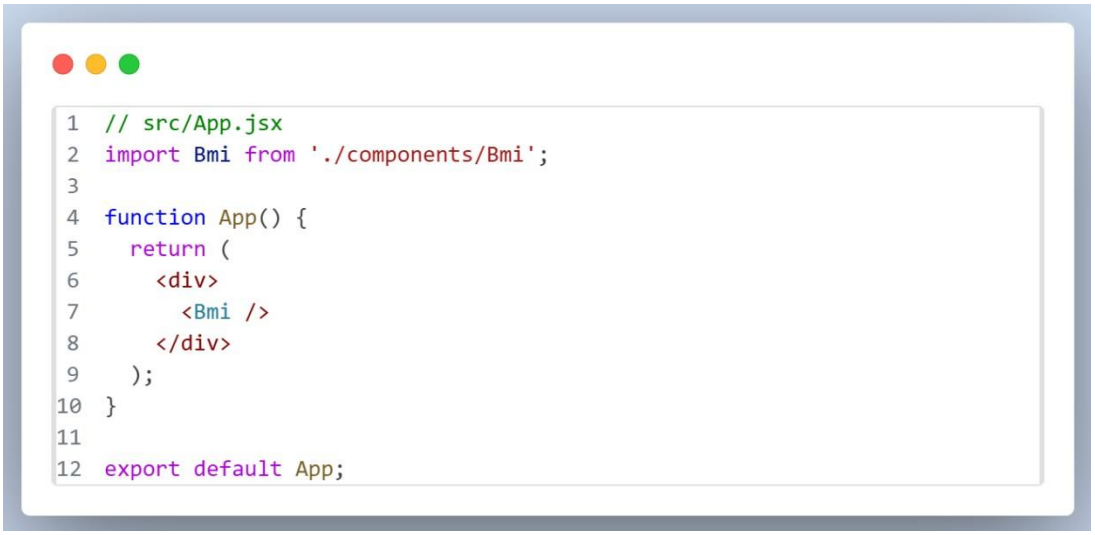
        <label className="label">
          Tinggi Badan (cm):
          <input
            type="number"
            className="input"
            value={tinggi}
            onChange={(e) => setTinggi(e.target.value)}
            placeholder="Masukkan tinggi badan"
          />
        </label>

        <button type="submit" className="button">Cek Kategori</button>
      </form>

      {/* Render Kondisional: Menampilkan hasil jika state 'hasil' tidak null */}
      {hasil && (
        <div className="result">
          {hasil.bmi > 0 && <p><strong>Nilai BMI:</strong> {hasil.bmi}</p>}
          <p><strong>Status:</strong> {hasil.status}</p>
        </div>
      )}
    </div>
  );
};

export default BmiCalculator;
```

13. Kemudian pada App.jsx tuliskan code sebagai berikut



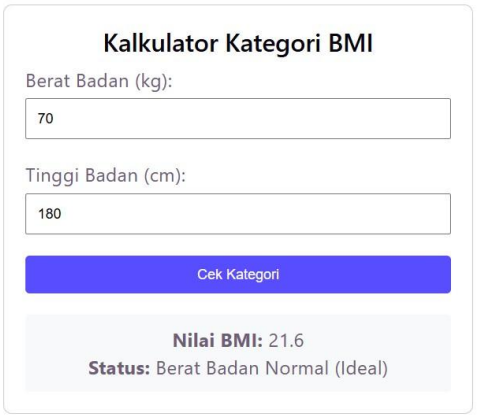
```jsx
// src/App.jsx
import Bmi from './components/Bmi';

function App() {
  return (
    <div>
      <Bmi />
    </div>
  );
}

export default App;
```

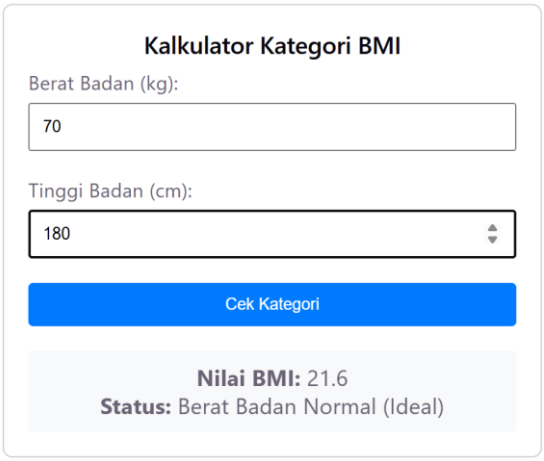
14. Hasil yang ditampilkan adalah sebagai berikut

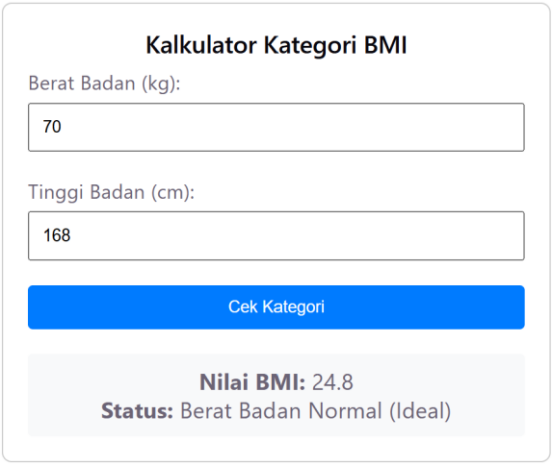






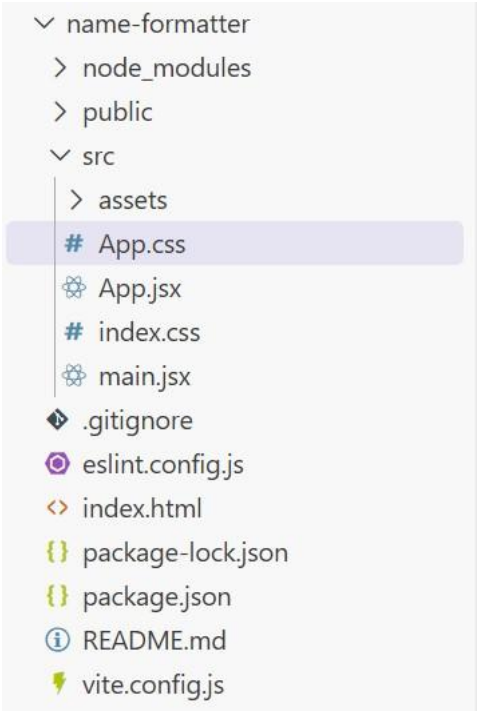






D. Program penjumlahan panjang nama dan jumlah suku kata nama

1. Lakukan proses yang sama seperti program sebelumnya (langkah 1-8) namun nama folder disesuaikan dengan nama folder name-formatter sampai bisa menghasilkan struktur folder seperti berikut.



2. Buat sub-folder components pada folder src. Kemudian buat file baru dengan nama NamaFormatter.jsx

```jsx
// src/components/NameFormatter.jsx
import { useState } from 'react';

const NameFormatter = () => {
  const [namaLengkap, setNamaLengkap] = useState('');
  const [hasil, setHasil] = useState(null);

  const handleFormat = (e) => {
    e.preventDefault();

    if (!namaLengkap.trim()) {
      alert('Harap masukkan nama lengkap terlebih dahulu!');
      return;
    }

    const namaKapital = namaLengkap.toUpperCase();
    const arrayKata = namaLengkap.trim().split(/\s+/);
    const inisial = arrayKata.map(kata => kata.charAt(0).toUpperCase()).join('');
    const gabunganFormat = `${inisial} - ${namaKapital}`;

    setHasil({
      namaKapital,
      inisial,
      gabunganFormat
    });
  };
```

```
27
28    return (
29      <div>
30        <h2>Format Nama & Inisial</h2>
31        <form onSubmit={handleFormat}>
32          <div>
33            <label>Masukkan Nama Lengkap: </label>
34            <input
35              type="text"
36              value={namaLengkap}
37              onChange={(e) => setNamaLengkap(e.target.value)}
38              placeholder="Contoh: Budi Santoso"
39            />
40          </div>
41          <br />
42          <button type="submit">Format Teks</button>
43        </form>
44
45        {hasil && (
46          <div>
47            <h3>Hasil Pemrosesan:</h3>
48            <p><strong>Nama Kapital:</strong> {hasil.namaKapital}</p>
49            <p><strong>Inisial Nama:</strong> {hasil.inisial}</p>
50            <hr />
51            <p><strong>Hasil Penggabungan:</strong> {hasil.gabunganFormat}</p>
52          </div>
53        )}
54      </div>
55    );
56  };
57
58  export default NameFormatter;
```

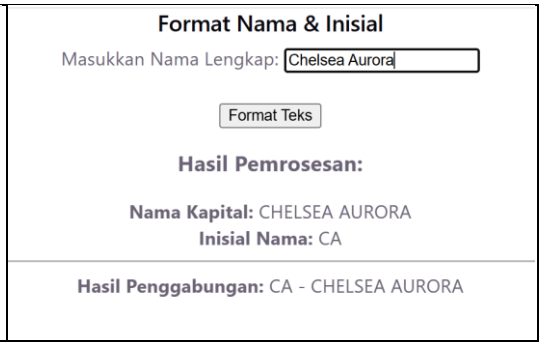
3. Kemudian pada App.jsx tuliskan seperti di bawah ini

```
1  // src/App.jsx
2  import NameFormatter from './components/NameFormatter';
3
4  function App() {
5    return (
6      <div>
7        <NameFormatter />
8      </div>
9    );
10 }
11
12 export default App;
```

4. Hasil yang diperoleh adalah sebagai berikut

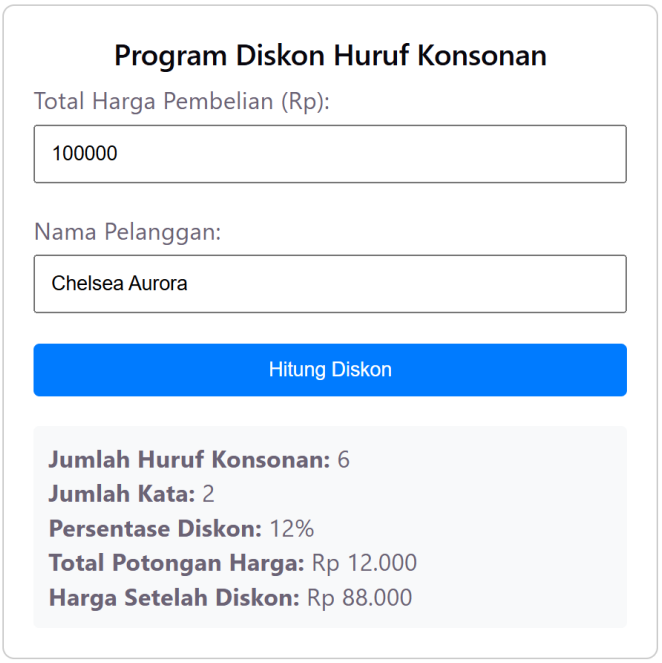
Format Nama & Inisial Masukkan Nama Lengkap: Contoh: Budi Santoso Format Teks	**Format Nama & Inisial** Masukkan Nama Lengkap: Muhammad Affan Format Teks **Hasil Pemrosesan:** **Nama Kapital:** MUHAMMAD AFFAN **Inisial Nama:** MA **Hasil Penggabungan:** MA - MUHAMMAD AFFAN
Before	**After**





5. Latihan

1. Buatlah program menggunakan Vite+Reactjs dengan struktur folder proyek yang baik untuk menghitung diskon belanja berdasarkan jumlah huruf konsonan dan jumlah kata pada nama pelanggan:
    - Judul : Program Diskon Huruf Konsonan
    - Input 1: Total Harga Pembelian
    - Input 2: Nama Pelanggan • Terdapat button hitung diskon
    - Ketentuan Logika:
    - Setiap huruf konsonan (selain A, E, I, O, U dan spasi) memberikan diskon sebesar 2% per karakter.
    - Jika nama terdiri lebih dari 2 kata (dipisahkan spasi), berikan bonus diskon tambahan 5%.
    - Maksimal total diskon yang bisa didapatkan adalah 20%.
    - Output yang Ditampilkan:
    - Jumlah Huruf Konsonan
    - Jumlah Kata
    - Persentase Diskon
    - Total Potongan Harga
    - Harga Pembelian Setelah Diskon



2. Buatlah Program menggunakan Vite+Reactjs dengan struktur folder proyek yang baik untuk mengecek apakah nama pelanggan merupakan sebuah *Palindrom* (kata yang dibaca sama dari depan maupun belakang, misal: "Anna", "Bob", "Madam") untuk menentukan promo.
    - Judul : Program Diskon Palindrom Nama
    - Input 1: Total Harga Pembelian
    - Input 2: Nama Pelanggan • Terdapat button cek & hitung
    - Ketentuan Logika:
    - Abaikan spasi dan perbedaan huruf besar/kecil (*case-insensitive*).
    - Jika nama termasuk *Palindrom* (dan minimal 3 karakter):
      Mendapatkan Diskon Spesial 25%.
    - Jika Bukan Palindrom:
      Diskon dihitung 1% per karakter unik (karakter yang sama tidak dihitung dua kali).
    - Ketentuan Logika:
    - Status Nama (Palindrom / Bukan Palindrom)
    - Jumlah Karakter Unik
    - Persentase Diskon
    - Harga Pembelian Setelah Diskon

Program Diskon Palindrom Nama

Total Harga Pembelian (Rp):

20000

Nama Pelanggan:

chelsea

Cek & Hitung

Status Nama: Bukan Palindrom
Jumlah Karakter Unik: 6
Persentase Diskon: 6%
Harga Setelah Diskon: Rp 18.800